

Query-Trials: three models write SQL

Youssef · Noura · Oumayma — CS496 Project 0 — run 20260915-103043 — generated 2026-09-15

1. The task

A model is given a database schema and a plain-English question, and must write one SQLite query that answers it. Scoring is **execution match**: we run the model's query and the reference query and compare the rows returned. The queries are never compared as text — two people can write the same query differently and both be right. No human judgement, no LLM judge.

Example item (q027, medium). What are the current job titles held by employee 10001?

Reference SQL: `SELECT title FROM titles WHERE emp_no = 10001 AND to_date = '9999-01-01' ORDER BY from_date;`

2. The data

The *Employees* sample database from Kaggle: six linked tables, 300,024 employees, 967,330 salary records. The dataset encodes "still current" as `to_date = '9999-01-01'` rather than `NULL`; several questions depend on it, and models that assume `NULL` get them wrong. That is legitimate difficulty, not a flawed question.

60 questions written by hand, all of them graded — the brief asks for at least 50. Every reference query was executed before any model ran, and its row count checked. Two items were replaced at that stage because they returned nothing by construction: one asked which departments have no current manager (all nine have one) and one asked a set to exceed its own maximum. Both were replaced **before** any model saw them — which is not the same as removing an item a model failed. Labels were double-checked by a second reader; see `data/labelling_note.md`.

3. Setup

Slot	Model	Exact name
Top API	Claude Opus 5	claude-opus-5
Cheap API	Claude Haiku 4.5	claude-haiku-4-5-20251001
Open-weights, self-hosted	Qwen2.5-Coder 3B (Q4_K_M)	qwen2.5-coder:3b-instruct-q4_K_M

Identical for all three: one prompt with the schema embedded (fingerprint printed on every result row), temperature 0, 300 output tokens, the same 60 items in the same order, one shared parser, calls made strictly sequentially, no retries. The three model adapters expose the same function signature, so the runner has no per-model branch — there is nowhere for a difference to hide.

Local model hardware — CPU: Intel(R) Core(TM) i5-8210Y CPU @ 1.60GHz, Memory: 8 GB, Model: MacBookAir8,2, OS: Darwin 23.6.0 (x86_64). Three warm-up calls were excluded from timing. We planned the 7B model but the available machine could not run it usefully; the 3B is reported as its own model, not as a stand-in.

4. Results

Model	Correct	Acc	p50	p95	Wrong	Err	\$/1k
qwen2.5-coder:3b-instruct-q4_K_M	21/60	35%	14281 ms	30827 ms	32	7	\$0.66

"Wrong" is a query that ran and returned different rows. "Err" counts SQL errors, timeouts, unsafe statements, unparseable output and failed API calls — all of which count as wrong, shown separately as the brief requires. Latency is time to the complete answer; we report p50 and p95 rather than the mean, because one very slow answer barely moves an average of fifty but is exactly what a user notices.

Wrong answers, and why

qwen2.5-coder:3b-instruct-q4_K_M

Item	Status	Why it failed	What it wrote
q007	wrong_result	returned 1 columns, expected 2	SELECT d.dept_name FROM departments d JOIN dept_emp de ON d.dept_no = de.dept_no GROUP BY d.dep
q010	wrong_result	same shape, different values	SELECT dept_no, COUNT(*) AS dept_manager_count FROM dept_manager GROUP BY dept_no ORDER BY dept
q019	wrong_result	same shape, different values	SELECT dept_name FROM departments WHERE dept_no LIKE 'd00%'

5. Our choice, and when it would change

TEAM: replace this paragraph with your own words before hand-in. The numbers above are generated; this judgement is not, and it is graded as judgement. State which model you would deploy, justify it with the accuracy gap against the cost gap in actual figures, then name the specific conditions that would change the answer — a higher accuracy bar, a volume past the break-even, a latency ceiling the local model cannot meet, or an offline requirement that rules out an API entirely.

6. Cost at 100× traffic, and break-even

API cost scales **linearly**: at 100× the traffic the bill is 100× larger, with no change in behaviour. Self-hosting is a **step function** — one machine absorbs the growth at almost no marginal cost until it saturates, at which point you buy another machine and the cost jumps. The two therefore cross at a specific volume.

Break-even is where total API spend equals total self-hosted spend, counting the one-off setup time as a fixed cost: $\text{api_per_1k} \times N/1000 = \text{fixed_setup} + \text{local_per_1k} \times N/1000$. Below that volume the API is cheaper overall; above it, self-hosting wins — but only if its p95 latency is acceptable, which for our machine is the binding constraint rather than the money. Exact figures and every assumption (hardware price, amortisation, electricity, utilisation) are printed by `python -m src.cost` and listed in `src/prices.yaml` with the date the prices were checked.

Reproduce: `python -m src.doctor && python -m src.run --model all && python -m src.score && python -m src.cost`